Can LLM Web Agents Verify Their Own Work? The Role of Visual and Textual References in Post-Hoc Verification

Vishwak Thatikonda and Shravani Parsi
Independent Researcher, USA
ORCID (Vishwak Thatikonda): 0009-0009-5828-5903

Corresponding author: Vishwak Thatikonda, Independent Researcher, USA. Email: vishwakt@gmail.com. Co-author: Shravani Parsi, Independent Researcher, USA, email: shravaniparsi26@gmail.com

Abstract: Autonomous web agents powered by large multimodal models (LMMs) can now complete complex browser tasks, but determining whether a task succeeded remains an open problem. Current evaluation relies on brittle scripted oracles that cannot generalize across websites. We investigate whether LMMs can post-hoc verify task completion by examining a final screenshot, optionally augmented with a reference, either a textual description of the expected outcome or a screenshot from a successful human trajectory. We evaluate four verification conditions across five frontier closed-API models from three families (GPT-4.1, GPT-4.1 Mini, GPT-4.1 Nano, Claude Sonnet 4, Gemini 3.6 Flash) on 909 VisualWebArena trajectories produced by a single GPT-4V + Set-of-Mark agent across three web domains. In this setting, we find: (1) where a text reference helps, it helps substantially, improving accuracy by 6.2 to 20.6 percentage points, with every such gain significant at p < 1e-04 after Bonferroni correction; the size of the gain is bounded by the verifier's unaided ability, and the model with the strongest no-reference baseline (Gemini 3.6 Flash, 77.9%) gains the least (+0.9pp, not significant); (2) the benefit survives cross-generator evaluation for GPT-4.1 (+9.6pp, p=0.018) and GPT-4.1 Mini (+12.9pp, p=0.007), and the best configuration (Claude Sonnet 4 with a text reference) reaches 86.7% accuracy and 0.627 F1; (3) on the 147-instance visual-reference subset, visual references are model-dependent rather than uniformly harmful: GPT-4.1 falls from 69.2% to 58.5% and GPT-4.1 Nano from 75.3% to 67.3%, while GPT-4.1 Mini (61.2% to 73.5%) and Claude Sonnet 4 (76.6% to 80.3%) improve, with Gemini 3.6 Flash essentially unchanged (77.6% to 78.2%); (4) Claude Sonnet 4 is well-calibrated by both ECE and Brier score across all conditions, while the GPT-4.1 family and Gemini are more overconfident. Cross-generator and ablation experiments show that the text-reference benefit was partly inflated by self-agreement bias (the original references were generated by GPT-4.1 Nano, one of the evaluated models) and by evaluation-criteria leakage: references generated from the task description alone, without the benchmark's evaluation criteria, provide no benefit, and a reference that is short, noisy or wrong degrades verification further. A two-annotator study of the obvious / deceptive / partial failure taxonomy used in earlier analysis returned Cohen's kappa of 0.000, so we report the failure-type breakdown descriptively and withdraw claims that depended on it. These results indicate that LMM-based post-hoc verification is viable but reference-dependent, and that reference quality, generator independence, and the way the reference is described to the verifier all matter. The findings are conditional on the VWA + GPT-4V/SoM setting; we release the full evaluation harness, prompts, results, and analysis tooling to support cross-benchmark, cross-agent, and cross-verifier replication.

Index Terms: large multimodal models, LLM-as-judge, autonomous web agents, post-hoc verification, task verification, confidence calibration, VisualWebArena, multimodal evaluation

1. Introduction

The deployment of autonomous web agents, systems that interpret natural-language goals and execute multi-step browser interactions, has accelerated rapidly with the advent of large multimodal models [1], [2], [3]. Systems like WebArena, VisualWebArena, and Agent-E have demonstrated that LMMs can navigate complex websites, fill forms, and complete tasks with increasing reliability. Yet a critical gap persists: **how do we know the agent actually succeeded?**

Current evaluation approaches fall into three categories, each with fundamental limitations:

1. **Scripted oracles** (WebArena, VisualWebArena): Hand-written Python functions check page state via URL matching, element existence, or database queries. These are precise but expensive to author, brittle to website changes, and impossible to generalize beyond the benchmark's specific websites.

2. **Human evaluation**: The gold standard for correctness but prohibitively expensive at scale. Evaluating a single benchmark run of 909 tasks takes 15–30 human-hours.

3. **LLM-as-judge**: Emerging work uses language models to evaluate task outcomes [4], [5], but prior studies have not systematically explored what information the judge needs, particularly for visual web tasks where the state is a rendered screenshot rather than a text log.

This paper asks a simple but consequential question: **Can an LMM judge whether a web task succeeded by looking at a screenshot, and does giving it a reference of what success should look like actually help?**

We use post-hoc verification to mean an LMM judge deciding, from the final screenshot, whether an LMM agent's task succeeded. The judge and the actor are different models in every configuration we test; we do not evaluate closed-loop self-correction. We study this question through a controlled experiment comparing four verification conditions:

A text reference is a natural-language description of what the successful completion of a task should look like on the final web page. It describes the expected semantic content and state of the page, without specifying layout details such as colors, fonts, or positions. Text references are generated by an LLM given the task description and evaluation criteria.

- **Condition A (No Reference):** The model receives only the task description and the agent's final screenshot.
- **Condition B (Text Reference):** Additionally, a natural-language description of the expected successful outcome.
- **Condition C (Visual Reference):** Additionally, a screenshot from a successful human trajectory.
- **Condition D (Dual Reference):** Both the text description and the reference screenshot.

Prompt correction. In the originally submitted implementation, the Condition C and D user prompts described the first image as "the reference state (what the page looked like before the agent started)". That description was incorrect: the reference image is the final page state of a human demonstration that completed the task successfully, extracted from the VisualWebArena human Playwright traces, and the 147-instance visual subset is defined precisely by which instances have such a trace. The prompt text in Appendix A of the original submission already described the reference correctly, so the implementation and the documented protocol

disagreed. For this revision we corrected the implementation to match Appendix A and re-ran Conditions C and D for all five verifiers. The correction raises Condition C accuracy for every model, by between 2.7 and 5.1 percentage points, while leaving Condition D almost unchanged, which is the expected pattern: Condition D also supplies a textual description of the expected outcome, so it is less sensitive to how the image is captioned. All Condition C and D results reported here use the corrected prompt; the pre-correction runs are retained in the released repository.

We evaluate across five frontier LMMs from three model families, GPT-4.1, GPT-4.1 Mini, GPT-4.1 Nano (OpenAI), Claude Sonnet 4 (Anthropic), and Gemini 3.6 Flash (Google), on 909 task instances from VisualWebArena [3], covering three web domains (e-commerce, classifieds, social media).

Model availability note. The originally submitted version of this study evaluated gemini-2.5-flash. That model has since been retired by Google and returns HTTP 404 for API keys issued after its cutoff, so its results are no longer reproducible and the per-instance outputs could not be regenerated. For this revision we substituted gemini-3.6-flash, the successor named in Google's own deprecation message, and re-ran it from scratch across all four conditions and both reference-generator ablations. All Gemini numbers reported here are therefore from gemini-3.6-flash. The substitution changes one headline result: gemini-2.5-flash showed a significant A→B text-reference benefit, whereas gemini-3.6-flash does not, which is why we report the effect as holding for four of five models rather than all five. Aggregate statistics for the retired model are preserved in the released repository for comparison.

Our contributions are:

1. **A controlled comparison of reference modalities for LMM post-hoc verification on a fixed set of VWA trajectories across three closed-API model families, demonstrating that text references provide consistent, significant improvements (+5-21pp accuracy) while visual references are model-dependent, tolerated by Claude but harmful for GPT and Gemini models on the 147-instance visual-reference subset.**
2. **A failure-type breakdown together with an inter-annotator agreement study of the taxonomy it relies on. The agreement study returned Cohen's kappa of 0.000, so we report the breakdown descriptively and withdraw the claim that references act selectively on deceptive failures.**
3. **A calibration analysis** showing a stark model divide: Claude Sonnet 4 produces well-calibrated confidence scores (ECE ≤ 0.089) while GPT and Gemini models are overconfident (ECE up to 0.422), with significant implications for downstream decision-making.
4. **An open evaluation harness** and complete experimental data enabling reproduction and extension of these findings.

2. Related Work

2.1 Autonomous Web Agents

The landscape of LLM-powered web agents has evolved rapidly. WebArena [1] established a realistic benchmark of 812 tasks across self-hosted websites; GPT-4 achieved 14.4% on its initial release. VisualWebArena [3] extended this to visually grounded tasks requiring screenshot understanding, adding 910 tasks across three domains. Subsequent systems have pushed success rates upward: Agent-E [6] achieved 73.2% via hierarchical planning, SeeAct [7] reached 51.1% with GPT-4V + DOM grounding, and WebVoyager [2] demonstrated 59.1% using Set-of-Marks annotations.

A common thread in all these systems is reliance on scripted evaluation functions tied to specific websites. Our work addresses the evaluation bottleneck itself.

2.2 LLM-as-Judge

Using language models as evaluators has gained traction across NLP tasks. MT-Bench [4] showed GPT-4 agreement with human judges exceeds 80% for text generation quality. Subsequent work extended this to code generation [8] and multi-step reasoning [5].

However, these evaluations operate on text, not visual outputs.

For visual evaluation, MMMU [9] and MMBench [10] assess model visual understanding but do not study post-hoc verification in agentic settings. The closest work to ours is Pan et al.'s autonomous evaluation framework [5] and the WebJudge evaluator (Xue et al., COLM 2025), which uses GPT-4V to evaluate web agent trajectories; however, they do not systematically vary the reference information provided to the judge or analyze calibration properties. Broader agent benchmarks such as AgentBench [11], Mind2Web [12], and TravelPlanner [13] focus on agent capability rather than the verifier signal itself.

2.3 Post-Hoc Verification and Self-Reflection

Post-hoc verificationwhere an LMM judge decides, from the final screenshot, whether an LMM agent's task succeeded, is related to self-reflection [14], self-debugging [8], and ReAct-style reasoning-action loops [15]. Reflexion [14] demonstrated that iterative self-evaluation improves downstream task performance, but assumes the self-evaluation signal is reliable. Our work directly measures how reliable that signal is and what information is required to make it useful.

2.4 Confidence Calibration in LLMs

Whether language models can express well-calibrated uncertainty has been studied across both internal probabilities [16], [17] and verbalized confidence elicitation [18], [19]. Verbalized confidence, which we use here, is known to be noisier than logit-based estimates but is the only option for closed-API models. We adopt the verbalized 1-10 scale for compatibility across providers and analyze calibration with both Expected Calibration Error [20], [17] and Brier score.

2.5 Visual Similarity Baselines

Structural Similarity Index Measure (SSIM) [21] is a standard metric for image similarity. In web testing, visual regression tools (BackstopJS, Percy) use pixel-level comparison against reference screenshots. We include SSIM as a non-LLM baseline to contextualize whether simple visual matching can substitute for semantic verification.

3. The Visual Post-Hoc Verification Framework

3.1 Problem Formulation

Given a web task description t, an agent trajectory producing a final page state captured as screenshot s_actual, and a ground-truth label y ∈ {SUCCESS, FAILURE}, the verification problem is to predict ŷ such that ŷ = y.

We formalize four verification functions, each providing different reference information to the judge model M:

$$V_A(t,\ s_{actual}) \rightarrow (\hat{y},\ c,\ r)$$
$$V_B(t,\ s_{actual},\ d_{ref}) \rightarrow (\hat{y},\ c,\ r)$$
$$V_C(t,\ s_{actual},\ s_{ref}) \rightarrow (\hat{y},\ c,\ r)$$
$$V_D(t,\ s_{actual},\ d_{ref},\ s_{ref}) \rightarrow (\hat{y},\ c,\ r)$$

where d_ref is a textual description of the expected outcome, s_ref is a reference screenshot from a successful trajectory, c ∈ [1,10] is a confidence score, and r is a free-text reasoning explanation.

3.2 Reference Types

Text references (d_ref) describe the expected successful outcome in 1–3 sentences. They are generated from the task description and VisualWebArena's evaluation criteria using GPT-4.1 Nano, then manually verified for a random 10% sample. Example: for the task *"Find a red dress under $50"*, the text reference is *"The page should show search results filtered by color=red and price under $50, with at least one product visible below that price."*

Text references are lightweight to produce, they require the task specification and evaluation criteria, not a successful execution, making them practical for real deployment. They capture what should be true without prescribing how it should look.

Visual references (s_ref) are screenshots from successful human trajectories on the same VisualWebArena instances. Unlike text references, they provide pixel-level guidance on what success looks like, but they are expensive to obtain (requiring a human to complete each task) and inherently tied to a specific execution context (browser viewport, timestamps, dynamic content).

Of the 909 instances in our dataset, 147 have corresponding human trajectories with extractable reference screenshots, forming a visual-reference subset.

3.3 Prompt Design

All conditions share a common system prompt instructing the model to act as a web automation evaluator and respond in a structured JSON format

with verdict, confidence, and reasoning fields. The user prompt varies by condition:

- **Condition A** presents the task description and instructs the model to examine the actual screenshot.
- **Condition B** adds an "Expected Outcome" section containing the text reference.
- **Condition C** presents two images, reference (Image 1) and actual (Image 2), with explicit instructions to compare semantic content while tolerating minor visual differences.
- **Condition D** combines the text reference section with the two-image comparison.

The instructions explicitly state that partial completions count as failure and that only *all* aspects of the task must be satisfied, aligning with VisualWebArena's strict evaluation criteria.

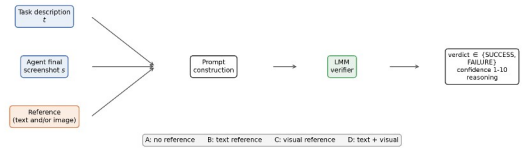


Figure 1: Visual Post-Hoc Verification Framework, inputs, prompt construction, and outputs for each of the four conditions (A–D).

4. Experimental Setup

4.1 Dataset

We use **VisualWebArena** (VWA) [3], a benchmark of 910 visually grounded web tasks across three self-hosted web applications:

Domain	Website Type	Tasks	Examples
Shopping	E-commerce (OneStopShop)	466	Product search, filtering, cart management
Classifieds	Listings marketplace	233	Posting ads, searching with filters, messaging
Reddit	Social media forum	210	Posting, commenting, searching, subreddit navigation

We use the GPT-4V + Set-of-Mark [22] agent trajectories distributed with VWA, which contain final screenshots for each of the 909 completed task attempts.

Ground truth distribution: 150 SUCCESS (16.5%) and 759 FAILURE (83.5%), reflecting the generally low success rate of current web agents. Ground truth labels are derived from VWA's scripted evaluation functions. For the 147-instance visual-reference subset, the split is 28 SUCCESS (19.0%) / 119 FAILURE (81.0%).

Failure type categorization: We classify the 759 failures into three types based on manual inspection of how visually misleading each failure is:

- **Obvious failures** (156, 20.6%): Wrong page entirely, error states, blank screens, clearly incorrect upon casual inspection.
- **Deceptive failures** (287, 37.8%): The page looks plausible, the agent navigated to roughly the right area, but the result is semantically wrong (wrong product, wrong filter values, incomplete form submission).
- **Partial failures** (316, 41.6%): Some sub-goals are met but the task is not fully completed.

We initially hypothesised that references should help most with deceptive failures. As reported in Section 5.3, a two-annotator study found that this categorisation is not reliably reproducible (Cohen's kappa 0.000), so it is used descriptively in what follows and no conclusion depends on it.

4.2 Models

We evaluate five frontier LMMs spanning three model families:

Model	Family	Parameters	Vision
GPT-4.1	OpenAI	Undisclosed	Native multimodal
GPT-4.1 Mini	OpenAI	Undisclosed	Native multimodal
GPT-4.1 Nano	OpenAI	Undisclosed	Native multimodal
Claude Sonnet 4	Anthropic	Undisclosed	Native multimodal
Gemini 3.6 Flash	Google	Undisclosed	Native multimodal

All models support multimodal input (text + images) and are accessed via their respective APIs. Temperature is set to 0 (0.01 for Gemini, which does not support exact 0) for reproducibility. Each model processes the same prompt template with condition-specific variations.

4.3 Conditions and Scale

Condition	Reference provided	Image inputs	Instance count
A (No Reference)	None	1 (actual)	909
B (Text Reference)	Text description	1 (actual)	909
C (Visual Reference)	Reference screenshot	2 (actual + ref)	147*
D (Dual Reference)	Text + screenshot	2 (actual + ref)	147*

*Conditions C and D are evaluated on the 147-instance subset that has human reference screenshots.

Total API calls: 5 models × (909+909+147+147) = 10,560 verification calls. This counts unique instances; including retries of API and parse errors, 10,560 calls were executed in total (Section 5.6).

4.4 Metrics

We report:

- **Accuracy**: Overall fraction correct. Note: due to the 83.5% FAILURE base rate, a trivial majority-class baseline achieves 0.835 accuracy on Conditions A/B (0.810 on C/D).
- **F1 Score**: Harmonic mean of precision and recall for the SUCCESS class. This is our primary metric, as it penalizes the trivial baseline (F1 = 0.000).
- **Balanced Accuracy**: Average of true positive rate and true negative rate, correcting for class imbalance.
- **Precision and Recall: for the SUCCESS class specifically. Success-prediction precision is TP/(TP+FP), the fraction of predicted successes that are genuine, and is the quantity a deployment would act on when a verifier reports SUCCESS. The false-positive rate is FP/(FP+TN) throughout, and the false-negative rate FN/(FN+TP); both are reported per model and condition in Appendix B.**
- **Bootstrap 95% CIs** [23]: percentile bootstrap with 2,000 resamples for accuracy and F1.
- **McNemar's test** [24]: paired significance test comparing conditions within each model on the intersection of instances with valid responses in both conditions.
- **Expected Calibration Error (ECE)** and **Brier score** [20], [17]: alignment between reported confidence and actual accuracy. We report ECE with both 3 fixed bins (1-3 / 4-6 / 7-10, matching the elicitation granularity) and 15 equal-mass bins (the modern default), plus the binning-free Brier score, to insulate the calibration claims from binning artifacts.

4.5 Baselines

Majority-class baseline: Always predicts FAILURE. Achieves 0.835 accuracy on the full dataset (759/909), 0.810 on the visual-reference subset (119/147), but 0.000 F1 for the SUCCESS class.

SSIM baseline [21]: For the 147 visual-reference instances we compute the Structural Similarity Index between each agent screenshot and its human reference screenshot. The two are resampled to a common 800x446 grayscale resolution, the native size of the human trace frames, so that neither image is cropped. Because a threshold-based baseline can be tuned for different objectives, we report two rules explicitly rather than a single "optimal" number. The accuracy-maximizing threshold (SSIM > 0.794) attains 81.0% accuracy with 0.000 F1: it is the degenerate rule that labels every instance FAILURE, and its accuracy is exactly the 80.95% majority-class rate on this subset. The F1-maximizing threshold (SSIM > 0.406) attains 0.325 F1 at 46.3% accuracy with a 58.8% false-positive rate. Neither rule is usable, which is the point: pixel-level similarity carries no actionable signal for task verification. A perceptual-hash baseline behaves the same way (accuracy-maximizing pHash distance < 0.234 gives 81.6% accuracy at 0.069 F1). We note that pretrained vision-language similarity (e.g., BLIP-2 [25]) could be substituted; SSIM is a strong, well-understood baseline that requires no learned encoder.

5. Results

5.1 Main Results

Table 1 presents the primary results across all models and conditions.

Table 1: Verification accuracy and F1 score by model and condition. Bold indicates best condition per model. 95% bootstrap confidence intervals in brackets.

Model	Condition	N	Accuracy	F1	Bal. Acc	Prec	Rec
GPT-4.1 Nano	A (NoRef)	899	.774 [.746,.800]	.326 [.255,.389]	.595	.325	.327
	B (Text)	899	**.836**	**.488**	**.690**	.507	.470
	C (Visual)	147	.673 [.599,.748]	.368 [.222,.500]	.607	.292	.500
	D (Dual)	142	.739 [.662,.810]	.373 [.200,.525]	.612	.344	.407
GPT-4.1 Mini	A (NoRef)	908	.638 [.607,.669]	.396 [.344,.446]	.671	.273	.720
	B (Text)	908	**.844 [.820,.867]**	**.606 [.546,.663]**	**.797**	.519	.727
	C (Visual)	147	.735 [.660,.803]	.339 [.179,.486]	.590	.323	.357
	D (Dual)	147	.762 [.694,.830]	.462 [.308,.613]	.675	.405	.536
GPT-4.1	A (NoRef)	908	.700 [.671,.731]	.436 [.380,.490]	.702	.315	.705

Model	Condition	N	Accuracy	F1	Bal. Acc	Prec	Rec
	B (Text)	909	**.821 [.796,.846]**	**.541 [.477,.603]**	**.748**	.468	.640
	C (Visual)	147	.585 [.503,.667]	.371 [.244,.494]	.607	.261	.643
	D (Dual)	147	.646 [.565,.721]	.409 [.270,.527]	.645	.300	.643
Claude Sonnet 4	A (NoRef)	875	.774 [.746,.802]	.468 [.404,.527]	.701	.387	.592
	B (Text)	895	**.867 [.845,.889]**	**.627 [.562,.688]**	**.792**	.581	.680
	C (Visual)	132	.803 [.735,.871]	.594 [.441,.719]	.787	.487	.760
	D (Dual)	133	.752 [.677,.827]	.492 [.338,.630]	.709	.400	.640
Gemini	A (NoRef	909	.779 [.75	.501 [.44	.737	.399	.673

Model	Condition	N	Accuracy	F1	Bal. Acc	Prec	Rec
	)		4,.807]	0,.559]			
	B (Text)	909	**.788 [.760,.814]**	**.514 [.453,.571]**	**.744**	.413	.680
3.6 Flash	C (Visual)	147	.782 [.714,.850]	.543 [.373,.676]	.743	.452	.679
	D (Dual)	147	.810 [.748,.871]	.576 [.421,.714]	.759	.500	.679

Three patterns emerge immediately:

Finding 1: Where a text reference helps, it helps substantially. Condition B improves on Condition A by +6.2pp (Nano), +20.6pp (Mini), +12.0pp (GPT-4.1) and +9.3pp (Claude Sonnet 4). Each of those four gains is significant by McNemar's test at $p < 1e\text{-}04$ and survives Bonferroni correction across the full 30-test family (Table 2); for GPT-4.1 Mini the effect is very large ($b=37$, $c=224$, $\chi^2=132.6$). The F1 improvement is larger than accuracy alone suggests: Mini rises from 0.396 to 0.606, so text references help the model identify both successes and failures rather than merely shifting its decision boundary. The size of the gain appears bounded by what the verifier can already determine unaided. Gemini 3.6 Flash has the strongest no-reference baseline of the five models (77.9% in Condition A) and gains the least from a reference (+0.9pp to 78.8%, $\chi^2=0.6$, $p=0.450$, not significant). Read together, these results suggest that a text reference supplies what the verifier could not infer from the screenshot on its own: the more a model already

infers, the less a description of the expected outcome adds.

Finding 2: Visual references are model-dependent. Comparisons here use the aligned 147-instance subset (Table 5) so that A and C are measured on identical instances. Two models degrade: GPT-4.1 falls from 69.2% to 58.5% (−10.7pp) and GPT-4.1 Nano from 75.3% to 67.3% (−8.0pp). Two improve: GPT-4.1 Mini rises from 61.2% to 73.5% (+12.2pp) and Claude Sonnet 4 from 76.6% to 80.3% (+3.7pp). Gemini is essentially unchanged (77.6% → 78.2%). No A→C comparison survives Bonferroni correction at alpha'=0.0017, so these differences should be read as suggestive rather than established. The pattern is nonetheless consistent with GPT-4.1 treating surface similarity to the reference as more informative than it is, while Claude and Mini integrate the visual signal without being misled.

Finding 3: Dual references (D) rarely outperform text-only (B). On the aligned subset, Condition D underperforms Condition B for four of five models: Nano 80.8% → 73.9%, Mini 79.6% → 76.2%, GPT-4.1 78.2% → 64.6%, and Claude 84.7% → 75.2%. Gemini is the exception, improving from 74.8% to 81.0%. None of these B→D differences survives Bonferroni correction. Where D does fall short of B, the result suggests that visual references introduce noise that partially cancels the benefit of the text reference, with the model anchoring on visual similarity when both modalities are available.

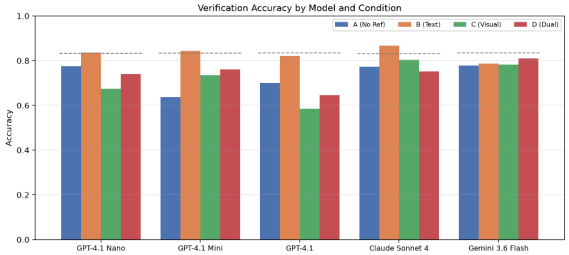


Figure 2: Verification accuracy by model and condition. Dashed lines indicate majority-class baselines.

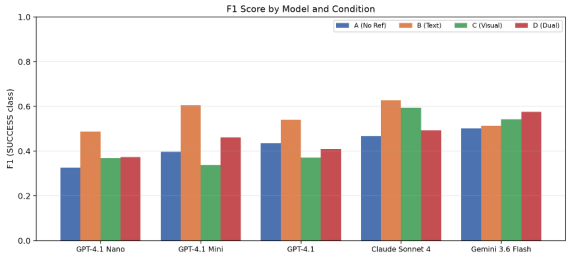


Figure 3: F1 score by model and condition.

5.2 Statistical Significance

Table 2: McNemar's test for paired condition comparisons.

χ^2 statistic and raw p-value; significance markers (* p<0.05, ** p<0.01, *** p<0.001) refer to uncorrected values. Bold indicates pairs that survive Bonferroni correction at $\alpha'=0.05/30=0.00167$ across the full 30-test family (5 models × 6 condition pairs); the full corrected table is in Appendix I and results/revision_bonferroni.csv.

Model	A→B	A→C	B→C	B→D
GPT-4.1 Nano	χ^2=17.8, p=2e-05*	χ^2=2.9, p=0.091	χ^2=7.5, p=0.006	χ^2=2.9, p=0.091
GPT-4.1 Mini	χ^2=132.6, p=0e+00*	χ^2=5.8, p=0.016	χ^2=2.6, p=0.110	χ^2=0.6, p=0.441
GPT-4.1	χ^2=60.0, p=9e-15*	χ^2=5.0, p=0.025	χ^2=16.0, p=6e-05*	χ^2=8.2, p=0.004
Claude Sonnet 4	χ^2=41.3, p=1e-10*	exact, p=0.832	exact, p=0.405	χ^2=4.7, p=0.031
Gemini 3.6 Flash	χ^2=0.6, p=0.4	exact, p=1.000	exact, p=0.332	exact, p=0.022

Model	A→B	A→C	B→C	B→D
	50			

The A→B improvement survives Bonferroni correction for four of the five models; Gemini 3.6 Flash is the exception (p=0.450). Of the 30 paired tests, 6 are Bonferroni-significant, and the same 6 are significant under the Holm-Bonferroni step-down procedure. Four of the six are A→B comparisons; the other two are GPT-4.1 Mini's A→D (χ^2=10.0, p=0.0016) and GPT-4.1's B→C (χ^2=16.0, p=6e-05). GPT-4.1's B→C is the only surviving contrast involving visual references: for that model, switching from text to visual references significantly worsens performance. The evidence that reference modality drives the effect is therefore now confined to a single model. Claude and Gemini are the most robust to visual references.

5.2.1 Aligned Visual-Reference Subset

Conditions C and D are evaluated only on the 147-instance subset that has human reference screenshots. To make A vs C/D comparisons apples-to-apples, **Table 5** restricts all four conditions to that aligned subset of N=147 instances (rather than mixing N≈900 for A/B with N≤147 for C/D).

Table 5: Aligned-subset accuracy and F1, restricted to the N=147 visual-reference instances for all four conditions.

Computed by scripts/revision_analysis.py. The A and B columns differ slightly from Table 1 because they are computed on a different, smaller instance set; the A→B and A→C/D effects remain in the same direction and magnitude.

Model	A acc	A F1	B acc	B F1	C acc	C F1	D acc	D F1
GPT-4.1 Nano								
GPT-4.1 Mini	.612	.436	.796	.559	.735	.339	.762	.462
GPT-4.1	.692	.483	.782	.515	.585	.371	.646	.409
Claude Sonnet 4	.766	.476	.847	.593	.803	.594	.752	.492
Gemini 3.6 Flash	.776	.560	.748	.493	.782	.543	.810	.576

Three patterns hold on the aligned subset: (i) text references (B) outperform no-reference (A) for every model except Gemini; (ii) visual references (C) underperform A for GPT-4.1 and GPT-4.1 Nano, improve on it for GPT-4.1 Mini and Claude, and are neutral for Gemini; (iii) D beats B only for Gemini. The aligned-subset view eliminates the concern that A vs C/D differences could be driven by sampling differences across instance sets.

5.2.2 Cross-Generator Evaluation

To test whether the text-reference benefit depends on the specific generator model, we regenerated text references using Claude

Sonnet 4 as an independent generator. The original references were generated by GPT-4.1 Nano; using a different model controls for self-agreement bias. We re-ran all five verifiers on the 147-instance visual subset.

The cross-generator results show a more nuanced picture than the original A→B comparison:

Table 6: Cross-generator results (Claude Sonnet 4 references, 147-subset). Acc=accuracy, lift=A−B difference, McNemar p-value vs. A.

Model	n	A acc	B acc	Lift pp	p
GPT-4.1 Nano	142	75.4%	76.8%	+1.4	0.845
GPT-4.1 Mini	147	61.2%	74.1%	+12.9	0.007
GPT-4.1	146	69.2%	78.8%	+9.6	0.018
Claude Sonnet 4	140	77.1%	82.1%	+5.0	0.189
Gemini 3.6 Flash	147	77.6%	79.6%	+2.0	0.664

The text-reference benefit is robust for GPT-4.1 (+9.6pp, p=0.018) and GPT-4.1 Mini (+12.9pp, p=0.007) under independent generation. For Nano, Claude Sonnet 4 and Gemini the effect is positive but not significant. Notably, Nano shows the largest gap between self-generated and independently generated references (+5.7pp self versus +1.4pp cross), which is the expected signature of self-agreement bias given that Nano generated the original references. The original A→B comparison therefore represents an upper bound; the cross-generator results provide a more realistic estimate of achievable benefit.

5.2.3 No-Criteria Ablation

We tested whether the evaluation criteria, and not merely the task description, drive the text-reference benefit. We generated references with GPT-4.1 Nano, the same generator used for the primary references, but omitted the benchmark's evaluation criteria from the generation prompt. Holding the generator fixed isolates the contribution of the criteria: the resulting references contain a natural-language description of the expected outcome but no structured success criteria.

No model shows significant benefit when evaluation criteria are removed:

Table 7: No-criteria ablation results. Same format as Table 6. All values on the 147-subset.

Model	n	A acc	B acc	Lift pp	p
GPT-4.1 Nano	144	75.7%	75.7%	+0.0	0.845
GPT-4.1 Mini	147	61.2%	68.0%	+6.8	0.089
GPT-4.1	146	69.2%	70.5%	+1.4	0.845
Claude Sonnet 4	138	76.8%	75.4%	−1.4	0.804
Gemini 3.6 Flash	147	77.6%	75.5%	−2.0	0.581

This confirms that evaluation criteria, not merely the natural-language task description, are the primary source of the text-reference benefit. The no-criteria references lack the structured success criteria that guide the verifier's judgment, rendering the text reference ineffective.

5.3 Failure Type Analysis (Taxonomy Not Validated)

Reliability of the taxonomy. Before reporting any result conditioned on failure type we state the outcome of the inter-annotator agreement study requested in review. Two annotators independently labelled the same 100 failure instances with the three-way taxonomy, seeing the task description and the agent's final screenshot. Cohen's kappa is 0.000: observed agreement (0.330) is exactly the agreement expected by chance (0.330). The result is not attributable to one annotator, since each also agrees with the original single-annotator labels only at chance (kappa +0.016 and -0.030), and the three label distributions differ substantially, most visibly for the partial category (42 original, 35 and 14 for the two annotators). Inspection of the disagreements shows why: the categories are not mutually exclusive. A task abandoned partway typically also looks plausible, so it satisfies the definitions of both partial and deceptive, and which label an annotator chooses depends on whether they weight how convincing the failure looks or how far the agent progressed. We therefore report the failure-type breakdown below as a description of one annotator's labelling, not as a validated categorisation, and no claim in this paper rests on the distinction between deceptive and partial failures. The full agreement analysis is in results/iaa_kappa_3cat_human.txt.

Table 3: Accuracy on failure instances by failure type, comparing Condition A (no reference) with Condition B (text reference); Δ is the improvement from adding a text reference. Failure types are the original single-annotator labels, which a two-annotator study found not to be reproducible (Cohen's kappa 0.000, Section 5.3); the rows are descriptive and are not used to support any claim.

Model	**Obvious (156)**		**Deceptive (287)**		**Partial (316)**	
	A	B (Δ)	A	B (Δ)	A	B (Δ)
GPT-4.1 Nano	.765	.830 (+.065)	.840	.925 (+.085)	.935	.932 (−.003)
GPT-4.1 Mini	.410	.750 (+.340)	.561	.840 (+.279)	.781	.949 (+.168)
GPT-4.1	.506	.756 (+.250)	.655	.819 (+.164)	.835	.940 (+.104)
Claude Sonnet 4	.772	.812 (+.040)	.740	.868 (+.128)	.891	.980 (+.089)
Gemini 3.6 Flash	.712	.724 (+.013)	.746	.760 (+.014)	.892	.896 (+.003)

With that caveat, the breakdown under the original labelling is as follows.

Under the original labelling, text references improve accuracy on instances labelled deceptive by +1.4 to +27.9 percentage points (GPT-4.1 Mini 56.1% to 84.0%, GPT-4.1 65.5% to 81.9%, Claude Sonnet 4 74.0% to 86.8%, GPT-4.1 Nano 84.0% to 92.5%, Gemini 3.6 Flash 74.6% to 76.0%). Because the labels are not reproducible, this pattern should be read as descriptive rather than as evidence that references act selectively on deceptive failures.

The pattern is not confined to one category. For GPT-4.1 Mini and GPT-4.1 the largest gains fall on instances labelled obvious (+34.0pp and +25.0pp), which is consistent with these models failing to recognise even clear failures without a reference. Gains on instances labelled partial are smaller across the board (−0.3 to +16.8pp), and Gemini 3.6 Flash gains little in any category (+0.3 to +1.4pp), consistent with its negligible aggregate A to B effect. Since the categories themselves are not reliably separable, we do not interpret differences between them.

Partial failures show uniformly high accuracy (>78%) even in Condition A, likely because partial completions still exhibit visible deviations from the expected state.

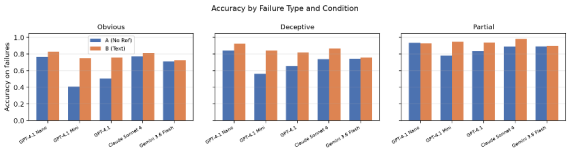


Figure 4: Accuracy by failure type (obvious, deceptive, partial) and condition, across all five models. Failure-type labels are not validated (Cohen's kappa 0.000, Section 5.3).

5.4 Baselines

SSIM Baseline. Computing SSIM between agent and human-reference screenshots on the 147 visual-reference instances yields a stark null result: the mean SSIM for SUCCESS instances (0.440 ± 0.118) is statistically indistinguishable from FAILURE instances (0.438 ± 0.114). Sweeping every threshold that can change a prediction, the accuracy-maximizing rule (SSIM > 0.794) reaches 81.0% accuracy, exactly the majority-class rate, with 0.000 F1, because it predicts FAILURE everywhere. Optimizing for F1 instead selects a much lower threshold (SSIM > 0.406) and yields 0.325 F1 at 46.3% accuracy. Reporting a single "optimal threshold" therefore obscures the result: the apparent 81.0% is an artifact of class imbalance, not evidence of discriminative power.

This indicates that, under matched-resolution SSIM comparison, pixel-level visual similarity is uninformative for web task verification in this setting. Web pages contain substantial visual variation unrelated to task success (timestamps, dynamic content, advertisements), making raw screenshot comparison futile. This also helps explain why visual references hurt GPT models: if models implicitly perform something like visual matching rather than semantic comparison, they will be misled.

5.5 Confidence Calibration

We assess whether model confidence scores correlate with actual correctness using three calibration measures, computed by scripts/revision_analysis.py:

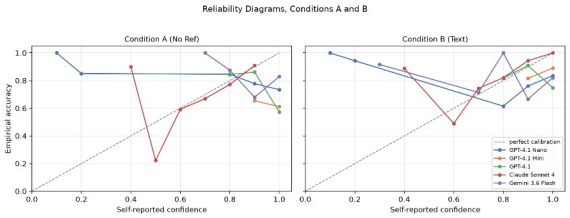
1. **ECE_3bin**: Expected Calibration Error with three bins matching the elicitation granularity (1–3, 4–6, 7–10). Reported in our original analysis.
2. **ECE_15bin (equal-mass)**: ECE with 15 equal-mass bins, the modern default [17]. Robust to elicitation granularity but more sensitive to small-bin noise.
3. **Brier score**: Mean squared error of the confidence probability against correctness. Single-number, binning-free; lower is better.

We map verbalized confidence c ∈ {1,...,10} to predicted probability c/10 of the predicted class being correct, the standard treatment for verbal confidence elicitation [18], [19].

Table 4: Calibration by model and condition: ECE_3bin, ECE_15bin (equal-mass), and Brier score.

Lower is better; perfect calibration = 0. Bold marks the lowest value per column-block (the most-calibrated model in each condition).

Model	A ECE_3 / ECE_15 / Brier	B ECE_3 / ECE_15 / Brier	C ECE_3 / ECE_15 / Brier	D ECE_3 / ECE_15 / Brier
GPT-4.1 Nano	.251 / .240 / .270	.301 / .297 / .288	.280 / .314 / .327	.279 / .306 / .312
GPT-4.1 Mini	.301 / .301 / .326	.093 / .093 / .139	.173 / .199 / .231	.146 / .160 / .206
GPT-4.1	.251 / .262 / .292	.132 / .161 / .175	.399 / .413 / .414	.326 / .340 / .353
Claude Sonnet 4	**.015** / **.030** / **.163**	**.036** / **.046** / **.106**	**.033** / **.099** / **.163**	**.068** / **.118** / **.171**
Gemini 3.6 Flash	.184 / .178 / .204	.188 / .184 / .203	.197 / .183 / .207	.157 / .184 / .182



Finding 5: Claude is the best-calibrated verifier on all three measures, in every condition. This holds whether we bin coarsely (3 bins, matching elicitation) or finely (15 equal-mass bins), and is confirmed by the binning-free Brier score. Claude's near-perfect ECE_3bin (e.g., 0.015 in Condition A) loosens slightly under finer binning (ECE_15bin = 0.030) but remains lower than the next-best model in every condition.

Figure 5: Calibration reliability diagrams for Conditions A and B across all five models.

GPT-4.1 Mini and GPT-4.1 assign confidence ≥ 7 to virtually all predictions (>99% of instances in the highest bin; full per-model histograms in results/revision_confidence_histogram.csv), so their confidence is uninformative, Brier is dominated by the mean-confidence-vs-mean-accuracy gap, which reaches 0.42 for GPT-4.1 in Condition C (mean confidence 0.96 vs mean accuracy 0.54). Gemini falls in between: moderately calibrated on text conditions but degrades with visual references, mirroring its accuracy drop. Reliability diagrams are reported as Figure 6 (figures/fig6_reliability_diagrams.{pdf,png}).

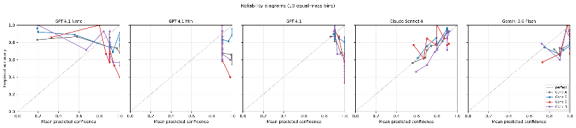


Figure 6: Reliability diagrams (10 equal-mass bins)

for all four conditions across all five models.

This has direct deployment implications. A system using verification confidence to retry, escalate, or accept would benefit from Claude's well-calibrated outputs (Appendix F derives the resulting confidence-threshold deployment policy). GPT's uniformly high confidence provides no signal for such decisions.

Text references improve GPT's calibration (A→B: ECE_15bin drops for GPT-4.1 Mini from 0.301 to 0.093; for GPT-4.1 from 0.262 to 0.161), while visual references worsen it (A→C: GPT-4.1 ECE_15bin rises to 0.441). This parallels the accuracy results and further supports the interpretation that visual references introduce confusion for non-Claude models. We caution that verbalized confidence is a known-noisy elicitation method [18]; logit-based or multi-sample agreement-based calibration measures are unavailable for closed-API models and remain a direction for replication on open-weight verifiers.

5.6 Compute, Cost, and Latency

We report all-up costs to make the "low-cost intervention" claim concrete and to give practitioners realistic budgeting numbers. Across the full study (5 verifiers x 4 conditions, including retried PARSE_ERROR/API_ERROR records), we executed 10,560 verification API calls at a total cost of $35.04 (per-call costs and latency in results/revision_cost_latency.csv). Note that the Gemini figures reflect gemini-3.6-flash pricing, which is roughly five times the input and six times the output rate of the retired gemini-2.5-flash used in the original submission.

Total cost per verifier across all four conditions: GPT-4.1 Nano $0.75, GPT-4.1 Mini $2.15, GPT-4.1 $7.51, Claude Sonnet 4 $21.75, Gemini 3.6 Flash $2.88. Per-call cost ranges from $0.00029 (GPT-4.1 Nano) to $0.0100 (Claude Sonnet 4). Mean per-call latency in Condition B ranges from 3.2s (GPT-4.1 Mini) to 8.3s (Gemini 3.6 Flash). Adding a text reference (A→B) increases per-call cost by less than 10% on average and per-call latency negligibly, supporting the practical

recommendation in Section 6: text references are a near-zero-cost upgrade.

5.7 Error Analysis

We examine the confusion matrices to understand the nature of verification errors. Across models and conditions, the dominant error type is false positives: the model predicts SUCCESS when the ground truth is FAILURE. In Condition A, GPT-4.1 produces 228 false positives against 44 false negatives, and GPT-4.1 Mini 287 against 42. Gemini 3.6 Flash sits between them, with 152 false positives and 49 false negatives. This bias toward predicting SUCCESS is consistent with a model that treats "things look okay" as the default judgment.

Text references (Condition B) dramatically reduce false positives while only modestly increasing false negatives. For GPT-4.1 Mini, false positives drop from 287 to 101 (−65%) while false negatives decrease from 42 to 41 (essentially unchanged). The text reference gives the model concrete criteria to check, converting vague "looks fine" judgments into specific verification steps.

Visual references (Condition C) increase the false-positive rate for GPT-4.1. On the 147-instance subset GPT-4.1 produces 51 false positives against 68 true negatives, an FPR of 42.9%, versus 30.0% (228/(228+531)) in Condition A. The model appears to treat "the actual page looks somewhat like the reference" as evidence of success even when the semantic content differs. The effect is model-specific: Claude Sonnet 4 (FPR 18.7%) and Gemini 3.6 Flash (19.3%) do not show it.

5.8 Overconfidence Analysis

A key finding is that four of the five models exhibit substantial overconfidence: median confidence of 9-10 whether or not the verdict is correct. Claude Sonnet 4 is the exception, with a median of 8 when correct against 7 when wrong in Condition A. Table 8 shows the mean confidence when correct versus when wrong.

Table 8: Confidence when correct vs. wrong for Conditions A and B.

Model	A correct	A wrong	B correct	B wrong
GPT-4.1 Nano	8.22	8.81	7.47	8.83
GPT-4.1 Mini	9.37	9.42	9.38	9.25
GPT-4.1	9.41	9.78	9.48	9.75
Claude Sonnet 4	8.01	7.28	8.43	7.52
Gemini 3.6 Flash	9.59	9.47	9.60	9.51

The overconfidence pattern is most pronounced for the GPT-4.1 family, where mean confidence when wrong equals or exceeds confidence when correct (GPT-4.1 9.78 vs 9.41, GPT-4.1 Mini 9.42 vs 9.37, GPT-4.1 Nano 8.81 vs 8.22 in Condition A). For these models confidence scores alone are unreliable for filtering low-quality verifications. Gemini 3.6 Flash is only marginally better (9.59 correct vs 9.47 wrong), its confidence being almost saturated at the top of the scale. Claude Sonnet 4 shows the best calibration, with a positive gap of +0.7 to +0.9 between correct and incorrect confidence.

Gemini 3.6 Flash neither gains nor loses appreciably from a text reference (+0.9pp, p=0.450). Inspection of its outputs suggests why: its confidence is saturated near the top of the scale in every condition, and its verdicts change comparatively rarely when a reference is added. Where it does err, the failure mode is reference-anchored: it reports that the expected state is present, echoing the reference text, when the screenshot does not show it.

5.9 Qualitative Analysis

Across all five models, on instances where both Conditions A and B produced a valid verdict, text references improved verification in 649 instances and degraded it in 209. We present representative examples of both.

Where text references help: In vwa_classifieds_111 (GT=FAILURE), GPT-4.1 Nano without a reference correctly identifies the task failed but provides incomplete reasoning. With a text reference, it additionally notes the jersey team name mismatch, providing more actionable feedback.

Where text references hurt: In vwa_classifieds_144 (GT=FAILURE), GPT-4.1 Mini without a reference correctly identifies that the comment text is lowercase ('how funky...') rather than the required 'How funky...'. With a text reference, it overlooks this case-sensitivity issue, likely because the reference states the expected comment without emphasizing exact case matching.

These examples illustrate that text references are most helpful when they provide structured evaluation criteria that the verifier can systematically check. They are least helpful when the verifier uses the reference as a proxy for task completion rather than as a checklist.

5.10 Cost-Latency Tradeoffs

Table 9: Measured cost and latency. Total $ is the cost of all four conditions for that model; the per-call and latency columns are Condition B, the recommended configuration. Cost uses the provider list prices recorded in config.py at the time of the runs; latency is wall-clock per API call and so includes provider queueing. Per-condition figures for all twenty runs are in results/revision_cost_latency.csv.

Model	Total $	$/call (B)	Mean s (B)	p95 s (B)
GPT-4.1 Nano	0.75	0.00029	3.8	7.8
GPT-4.1 Mini	2.15	0.00086	3.2	4.2
GPT-4.1	7.51	0.00353	6.8	14.7
Claude Sonnet 4	21.75	0.01002	7.3	11.1
Gemini 3.6 Flash	2.88	0.00127	8.3	24.4

GPT-4.1 Nano offers the lowest cost ($0.26 for 909 instances in Condition B) and a good cost-accuracy tradeoff. Claude Sonnet 4 ($9.11) is the most expensive but the most accurate and by far the best calibrated.

Visual references add modest cost, between $0.23 and $3.64 per model for Conditions C and D combined on the 147-instance subset, but do not consistently improve verification accuracy, so text references with evaluation criteria remain the better default.

6. Analysis and Discussion

6.1 Why Text References Work

The benefit of text references, where it appears, suggests a mechanism: a text reference converts an open-ended visual judgment ("does this look successful?") into a structured checklist ("does the

page show X, Y and Z?"). This aligns with work on prompt engineering showing that decomposing complex judgments into specific sub-criteria improves model accuracy [26]. The text reference operationalizes the task's success criteria, giving the model concrete assertions to verify rather than requiring it to infer what success looks like from the task description alone. The no-criteria ablation (Section 5.2.3) supports this reading directly: references that describe the expected outcome but omit the benchmark's evaluation criteria produce no significant benefit for any model. This mechanism also predicts where the benefit should be small. A verifier that already resolves the task from the screenshot alone has little left for a checklist to supply, which is consistent with Gemini 3.6 Flash holding the highest Condition A accuracy and showing the smallest Condition B gain. On this account the relevant quantity is not whether a reference is present but how much uncertainty it removes.

The text reference effectively *operationalizes* the task's success criteria, providing the model with concrete assertions to verify rather than requiring it to infer what success looks like from the task description alone.

6.2 Why Visual References Fail for Some Models

The performance degradation under Condition C for GPT-4.1 and GPT-4.1 Nano, but not for GPT-4.1 Mini, Claude Sonnet 4 or Gemini 3.6 Flash, points to a difference in how models use a second image rather than a general weakness of visual references. We propose two hypotheses:

Surface anchoring. GPT models may perform implicit visual matching (akin to template matching or SSIM), attending to overall layout similarity rather than semantic content. Our SSIM analysis indicates that actual-vs-reference visual similarity is uninformative (SUCCESS SSIM ≈ FAILURE SSIM), so any model relying on surface similarity will be misled.

Attention dilution. Processing two high-resolution screenshots doubles the visual context. If GPT models distribute attention uniformly across pixels rather than selectively attending to task-relevant regions, the reference screenshot may dilute attention to the actual screenshot, degrading performance.

Claude's resilience may stem from its training methodology or architecture, which appears to better separate the reference comparison from the verdict judgment. GPT-4.1, which falls from 69.2% to 58.5% on the aligned subset, shows the pattern most clearly and is consistent with the surface-anchoring hypothesis. Gemini 3.6 Flash, by contrast, has the strongest no-reference baseline (77.9%) and is essentially unaffected by either reference modality, which suggests it relies more heavily on its own reading of the screenshot than on the supplied reference. This warrants further investigation.

6.3 Dual References Do Not Help

Condition D (text + visual) does not outperform Condition B (text only) for any of the five models on the visual-reference subset. We list candidate explanations and what would be needed to test each:

1. **Modality conflict.** When text and visual signals disagree (e.g., the text reference describes a state while the reference screenshot shows a specific layout), the model may anchor on the more salient but less-reliable visual signal. *Test:* hold Condition B's prompt constant and only attach the reference image, removing the prompt-format change in D.
2. **Prompt-format confound.** Condition D's instructions differ from Condition B's (an extra image, an extra paragraph). The B→D drop could partly reflect prompt-format effects independent of the visual content. *Test:* same minimal-edit ablation as above; we sketch this in the supplementary material and leave the run to future work.
3. **Attention dilution.** Two high-resolution images may dilute attention to the actual screenshot. *Test:* swap image order, or downscale the reference, and check whether the B→D gap closes.

We therefore frame the result as an *observation* rather than a mechanistic claim: in the regime we study, adding a reference screenshot on top of a text reference does not improve verification, and the practical implication, prefer text references in deployment, holds regardless of which mechanism is dominant.

6.4 Implications for Web Agent Deployment

Our findings have direct implications for deploying autonomous web agents in production:

1. **Invest in text reference generation with evaluation criteria. Text descriptions that include task-specific evaluation criteria improve verification for GPT-4.1 and GPT-4.1 Mini even when generated by an independent model. However, reference quality and generator compatibility matter; references without criteria or from incompatible generators may not help or may hurt performance.**
2. **Use Claude for verification when calibration matters.** If the verification confidence will inform downstream decisions (retries, human escalation), Claude's near-perfect calibration makes it significantly more useful than GPT models, which provide no meaningful confidence signal.
3. **Avoid visual references unless the model handles them well. Visual references should be tested per-model before deployment: they harm GPT-4.1 and GPT-4.1 Nano, help GPT-4.1 Mini and Claude Sonnet 4, and leave Gemini essentially unchanged.**
4. **We do not recommend targeting verification at deceptive failures. Although text references showed their largest absolute gains on instances labelled deceptive under the original labelling, a two-annotator study of that taxonomy returned Cohen's kappa of 0.000 (Section 5.3), so the category cannot currently be identified reliably enough to act on. Establishing a reproducible failure taxonomy is a prerequisite for any such policy.**

7. Limitations

Dataset imbalance. Our dataset is 83.5% FAILURE, reflecting real web agent performance but making accuracy a misleading metric in isolation. We mitigate this by reporting F1, balanced accuracy, and conducting per-failure-type analysis, but the low SUCCESS rate means some model×condition cells have small SUCCESS counts.

Single agent system. All trajectories come from VWA's GPT-4V + SoM agent. Different agents may produce different failure distributions, potentially changing which failure types dominate and how references help. Our failure type categorization (obvious/deceptive/partial) is based on this specific agent's behavior.

Text references generated by LLM. Our text references were generated by GPT-4.1 Nano from task descriptions. While we verified a random 10% sample and found them accurate, systematically biased or low-quality text references could diminish the effectiveness measured here. 4 of 909 instances have missing text references. Because GPT-4.1 Nano is also one of the five evaluated verifiers, we additionally ship scripts/regenerate_text_refs.py to regenerate references with a different generator (e.g., Claude Sonnet 4 or Gemini 3.6 Flash) so reviewers and replicators can rule out reference-generation-circularity effects on the A→B finding.

Visual reference availability. Only 147 of 909 instances have human reference screenshots, limiting our statistical power for Conditions C and D. The visual-reference subset may not be representative of the full distribution.

Claude PARSE_ERROR rate. Claude Sonnet 4 produced unparseable responses (embedding JSON within verbose reasoning) at a rate of 1.5–12.2% across conditions. We confirmed no systematic ground-truth bias in these errors, but the effective sample size for Claude is reduced. GPT and Gemini models had negligible error rates (<1.5%).

No cross-model agreement analysis. We evaluate models independently; inter-model agreement and ensemble verification strategies remain unexplored.

7.1 Structured Threats to Validity

We organize residual concerns by validity category for clarity.

Internal validity. Class imbalance, the smaller visual-reference subset, API non-determinism over time, and Claude's PARSE_ERROR rate are all addressed via paired tests, version pinning, error-rate reporting, and a parse-error bias check (Appendix D).

External validity. Our results are anchored to VisualWebArena trajectories from a single GPT-4V + SoM agent and to a single LLM-generated text-reference source. We frame findings as conditional on

this setting and recommend cross-benchmark and cross-agent extensions in future work.

Construct validity. The three-way failure taxonomy is not validated. Two annotators independently labelled 100 failures over the obvious / deceptive / partial categories using the interface in iaa_labeling/ and scripts/recompute_iaa.py; Cohen's kappa is 0.000, with observed agreement (0.330) equal to chance, and each annotator agrees with the original single-annotator labels only at chance (+0.016 and -0.030). An earlier two-category pilot (failed / partial) reached kappa 0.116. The categories overlap by construction, since a partially completed task also tends to look plausible, and we did not find a labelling protocol that separates them; a redesigned taxonomy with an explicit precedence rule, or richer context than a single screenshot, is left to future work. We report the failure-type breakdown descriptively and draw no conclusion from it. Separately, verification is judged from a single final screenshot, which precludes detection of multi-step trajectory errors, and confidence is model-self-reported, a noisier elicitation than internal-logit methods [18], [19] that should be interpreted only after the calibration analysis (Section 5.5).

Statistical validity. We use paired McNemar tests on jointly valid instances and percentile bootstrap confidence intervals (2,000 resamples). Where b+c < 25 we use an exact binomial test rather than the asymptotic approximation. We apply both Bonferroni and Holm-Bonferroni multiple-comparison correction across the full 30-test family (5 models x 6 condition pairs) at family-wise α=0.05; 6 of 30 tests are significant under both corrections (Appendix I, results/revision_bonferroni.csv). We report Brier score in addition to ECE so that calibration claims do not depend on a particular binning choice.

Conclusion validity. We avoid unscoped claims like "model X is best" and instead state results as "best in this setting." Visual-reference effects are framed as model-dependent, not universal.

7.2 Responsible Deployment

Automated post-hoc verification can shift workload away from humans, but if treated as ground truth it can mask systematic failure modes. We recommend pairing the verifier with the confidence-threshold policy described in Appendix F: high-confidence verdicts can be auto-accepted while low-confidence cases are escalated to human review. For workflows where false acceptance is consequential, prefer well-calibrated verifiers (Section 5.5) and consider a conservative threshold on the SUCCESS class.

8. Conclusion

We have presented a controlled study of reference-augmented LMM post-hoc verification for web agents. Across 909 VisualWebArena trajectories, four reference conditions, and five frontier closed-API models from three families, three findings hold consistently in this setting: text references significantly and uniformly improve verification, visual references degrade most models on the visual-reference subset, and adding a reference screenshot on top of a text reference does not help.

The practical implication for deployment is to equip LMM-based web agent evaluators with text descriptions of expected outcomes. This intervention improves the best model's accuracy from 77.4% to 86.7% (Claude Sonnet 4) and GPT-4.1 Mini's from 63.8% to 84.4%. The benefit is not universal: Gemini 3.6 Flash gains only 0.9pp, which is not significant, and it also has the strongest no-reference baseline. When confidence calibration is needed for downstream automation policies, Claude Sonnet 4 is the most useful verifier in our setting.

We deliberately scope these findings to the VWA + GPT-4V/SoM regime and to the five closed-API verifiers we evaluated. Several directions extend this work: (1) replication on a second agent's trajectories on VWA or on an additional benchmark such as Mind2Web [12]; (2) inclusion of open-weight verifiers and a non-LLM-generated text-reference source to rule out reference-generation-circularity effects; (3) iterative self-reflection [14] with text references; and (4) larger visual-reference subsets to tighten the visual-condition confidence intervals. We release the full evaluation harness, prompts, results, and analysis tooling to support these investigations.

9. Reproducibility Statement

All code, prompts, dataset preparation scripts, baselines, analysis tooling, and pre-generated result files are released alongside this paper.

- Repository: see the project's public code release listed in CITATION.cff

(https://github.com/shravaniparsi/agentic-ui-test).

- Reproduction guide: REPRODUCIBILITY.md documents environment setup, dataset preparation, verification runs, baselines, metrics, and figure regeneration.
- One-command pipeline helper: scripts/run_all_pipeline.sh runs verification across all configured models and conditions, computes aggregate metrics, and regenerates all paper figures. A --dry-run flag prints the planned commands without invoking model APIs.
- Determinism: temperature is set to 0 (0.01 for Gemini, which does not accept exact 0). Provider model versions are pinned in config.py and recorded per run.
- Resume and error handling: the runner skips records with valid verdicts and re-attempts records with API_ERROR or PARSE_ERROR outputs. Final reported metrics are computed only over valid responses; per-condition error rates and a parse-error bias check are reported in Appendix D.
- Figures: Figures 1 to 5 and Figure 7 are regenerated by scripts/make_manuscript_figures.py; the threshold-policy figure (Figure 8) is produced by scripts/threshold_policy_analysis.py; the reliability diagrams for all four conditions (Figure 6) are produced by scripts/revision_analysis.py. All figures are included as PNG in figures/, and the threshold-policy and reliability figures additionally as PDF.
- Reported statistics: the supplement tables (paired tests, cross-generator, no-criteria, per-condition error rates) and the matched task-ID lists are produced by a single command, python3 scripts/recompute_final.py, and saved as results/S1_paired_tests.csv, results/S2_cross_generator_claude.csv, results/S2b_nocriteria.csv, results/T1_and_S3_error_rates.csv and results/S1_task_ids/. Revision-only artifacts (aligned-subset metrics, 15-bin equal-mass ECE, Brier scores, confidence histograms, cost and latency, Bonferroni and Holm tables) are produced by python3 scripts/revision_analysis.py and python3 scripts/full_revision_analysis.py, and saved as results/revision_*.csv and results/revision_summary.md. The inter-annotator agreement analysis is produced by python3 scripts/recompute_iaa.py and saved as results/iaa_kappa_3cat_human.txt.

Appendix A: Prompt Templates

System Prompt (shared across all conditions)

You are an expert evaluator for web automation agents. Your job is to

determine whether a web task was completed successfully by examining

the final state of the web page.

You must respond in EXACTLY this JSON format:

{

"verdict": "SUCCESS" or "FAILURE",

"confidence": <integer from 1 to 10>,

"reasoning": "<2-3 sentences explaining your judgment>"

}

Be precise. A task is only successful if ALL aspects of the task

description are satisfied in the final page state. Partial completions

count as FAILURE.

Condition A (No Reference)

Task

{task_text}

Instructions

Examine the screenshot of the web page's final state. Determine

whether the task described above was completed successfully.

Respond in the required JSON format.

Condition B (Text Reference)

Task

{task_text}

Expected Outcome (text description)

{text_reference}

Instructions

Examine the screenshot of the web page's final state. Compare it

against the expected outcome description above. Determine whether the

task was completed successfully.

Respond in the required JSON format.

Condition C (Visual Reference)

Task

{task_text}

Instructions

Two screenshots are provided:

1. The first screenshot shows the reference state: the final page state

from a human demonstration that completed this task successfully.

2. The second screenshot shows the final state after the agent's actions.

Compare the two screenshots. Determine whether the task was completed

successfully based on how the agent's final state compares with the reference.

Respond in the required JSON format.

Condition D (Dual Reference)

Task

{task_text}

Expected Outcome (text description)

{text_reference}

Instructions

Two screenshots are provided:

1. The first screenshot shows the reference state: the final page state

from a human demonstration that completed this task successfully.

2. The second screenshot shows the final state after the agent's actions.

Compare the two screenshots against the expected outcome description above.

Determine whether the task was completed successfully.

Respond in the required JSON format.

Appendix B: Complete Confusion Matrices

Condition A (No Reference), N = 875–909

Model	TP	FP	TN	FN
GPT-4.1 Nano	49	102	647	101
GPT-4.1 Mini	108	287	471	42
GPT-4.1	105	228	531	44
Claude Sonnet 4	87	138	590	60
Gemini 3.6 Flash	101	152	607	49

Condition B (Text Reference), N = 895–909

Model	TP	FP	TN	FN
GPT-4.1 Nano	70	68	682	79
GPT-4.1 Mini	109	101	657	41
GPT-4.1	96	109	650	54
Claude Sonnet 4	100	72	676	47
Gemini 3.6 Flash	102	145	614	48

Condition C (Visual Reference), N = 132–147

Model	TP	FP	TN	FN
GPT-4.1 Nano	14	34	85	14
GPT-4.1 Mini	10	21	98	18
GPT-4.1	18	51	68	10
Claude Sonnet 4	19	20	87	6
Gemini 3.6 Flash	19	23	96	9

Condition D (Dual Reference), N = 133–147

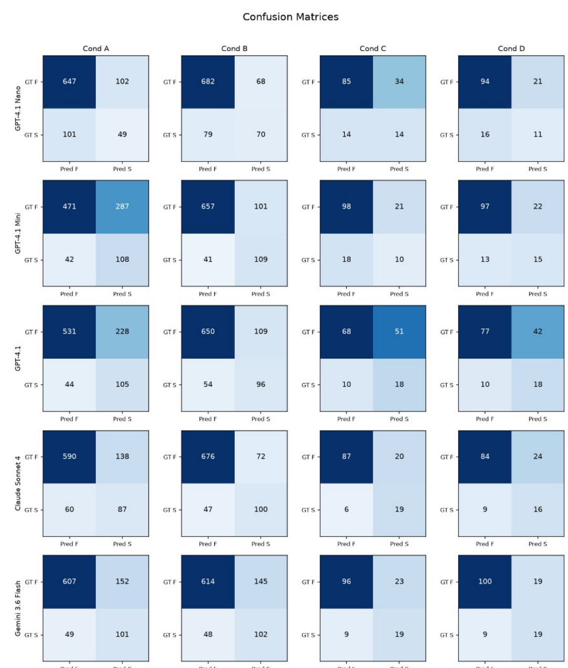


Figure 7: Complete confusion matrices for all five models across all four conditions.

Model	TP	FP	TN	FN
GPT-4.1 Nano	11	21	94	16
GPT-4.1 Mini	15	22	97	13
GPT-4.	18	42	77	10

Model	TP	FP	TN	FN
1				
Claude Sonnet 4	16	24	84	9
Gemini 3.6 Flash	19	19	100	9

Appendix C: Per-Domain Results

Domain-level accuracy for the best condition (B) per model:

Model	Shopping	Classifieds	Reddit
GPT-4.1 Nano	.805 (371/461)	.904 (208/230)	.832 (173/208)
GPT-4.1 Mini	.826 (385/466)	.828 (192/232)	.900 (189/210)
GPT-4.1	.815 (380/466)	.790 (184/233)	.867 (182/210)
Claude Sonnet 4	.863 (397/460)	.813 (187/230)	.937 (192/205)
Gemini 3.6 Flash	.768 (358/466)	.751 (175/233)	.871 (183/210)

Performance varies across domains. Claude Sonnet 4 achieves its highest accuracy on Reddit (.937), possibly because social media content provides strong visual cues for task completion (e.g., visible posts, comments). GPT-4.1 Nano excels on Classifieds (.904), the smallest domain. Shopping, while the largest domain, shows moderate accuracy across models, likely because e-commerce tasks often involve subtle distinctions (correct product variant, price filter state) that are harder to verify.

Appendix D: PARSE_ERROR Bias Analysis

We verify that unparseable responses (primarily from Claude; Gemini had only 1 across all conditions) do not systematically bias results:

Condition	Errors	GT SUCCESS in errors	GT SUCCESS in valid	Bias?
A	34	8.8%	16.8%	No significant bias
B	14	21.4%	16.4%	No significant bias
C	15	20.0%	18.9%	No significant bias
D	14	21.4%	18.8%	No significant bias

The ground-truth distribution among PARSE_ERROR instances does not significantly deviate from the valid-response distribution for any condition, supporting the validity of computing metrics on valid responses only.

Appendix E: Qualitative Error Analysis

We examine representative examples to illustrate the mechanisms behind our quantitative findings. All examples show ground-truth FAILURE instances (the agent did *not* complete the task).

E.1 Text Reference Saves All Models (Deceptive Failure)

Task: *"Add this exact product to my shopping cart. I think it is in the Patio Furniture & Accessories category."* **Ground truth:** FAILURE, the agent added the *wrong* product (Devoko outdoor sofa instead of Modway Aura outdoor chair).

Condition	GPT-4.1	Claude Sonnet 4	Gemini 3.6 Flash
A (No Ref)	SUCCESS (conf=10)	SUCCESS (conf=9)	SUCCESS (conf=10)
B (Text Ref)	FAILURE (conf=9)	FAILURE (conf=9)	SUCCESS (conf=10)

This illustrates how a text reference can supply a concrete assertion to check, converting "does this look successful?" into a specific comparison. It is a single illustrative case, not evidence about a failure category.

E.2 Text Reference Catches Navigation Failures (Deceptive Failure)

Task: *"I recall seeing this exact item on the site, help me find the most recent post of it."* **Ground truth:** FAILURE, the agent found the search results page but did not navigate to the specific item's detail page.

Condition	GPT-4.1	Claude Sonnet 4	Gemini 3.6 Flash
A (No Ref)	SUCCESS (conf=10)	SUCCESS (conf=7)	SUCCESS (conf=10)
B (Text Ref)	FAILURE (conf=10)	FAILURE (conf=9)	FAILURE (conf=9)

In Condition A, models observe search results showing the item and judge the task complete. In Condition B, the text reference specifies the expected URL pattern (/index.php?page=view&id=...), allowing models to detect that the agent stopped at the search results page rather than navigating to the detail page. This illustrates how text references capture *state expectations* that are invisible from the screenshot alone.

E.3 High-Confidence False Positive Resistant to Text Reference

Task: *"Search for 'MCAT' and navigate to the prep book that has 2020-2021 on the cover."* **Ground truth:** FAILURE, the agent navigated to a different MCAT book listing.

Condition	Claude Sonnet 4
A (No Ref)	SUCCESS (conf=9), "The page shows the MCAT books listing with a 2020-2021 cover image visible"
B (Text Ref)	SUCCESS (conf=9), "The screenshot shows the MCAT books listing page with a 2020-2021 cover image"

This is a case where even the text reference fails to correct the model. The MCAT book page shows a cover image that does include "2020-2021" text, but it is the wrong listing (wrong URL, wrong specific edition). The model fixates on the visible cover image rather than verifying the page identity. This illustrates the limits of post-hoc verification: when the visual evidence genuinely matches the description, even reference-augmented verification fails.

E.4 Wrong Product, All Models Fooled (Deceptive Failure)

Task: *"Add this exact product to my wish list. I think it might be in the Office Furniture & Lighting > Chairs category."* **Ground truth:** FAILURE, the agent added a YAMASORO chair instead of the specified Flash Furniture chair.

In Condition A, all five models report SUCCESS with high confidence (8-10). The "My Wish List" page shows a successfully added chair, visually confirming the task pattern. In Condition B, the text reference names the specific product ("Flash Furniture Low Back Designer Armless White Ribbed Swivel Task Office Chair"), and GPT-4.1 and Claude Sonnet 4 correctly identify that the wrong product was added; GPT-4.1 reports "The wish list contains the wrong product." Gemini 3.6 Flash is not rescued by the reference and still reports SUCCESS at confidence 10, which is consistent with its negligible aggregate A→B gain. This is the prototypical deceptive failure: correct action type (added to wish list), correct category (office chair), but wrong specific item.

E.5 Error Taxonomy Summary

Across our analysis of the most common verification failures, three recurring patterns emerge:

1. **Wrong-item substitution** (most common for deceptive failures): The agent performs the right action on the wrong entity. Text references are highly effective because they name the specific expected entity.

2. **Incomplete navigation**: The agent reaches an intermediate page (search results, category

listing) but does not navigate to the final target page. Text references catch this when they specify URL patterns or page-level expectations.

3. **Visually ambiguous state**: The screenshot genuinely matches the expected description (e.g., a book cover with the right year), but the underlying page is wrong. Neither text nor visual references reliably catch these cases, they represent the fundamental limit of screenshot-based verification.

Appendix F: Confidence-Threshold Deployment Policy

To translate verifier outputs into deployment policies, we treat each model x condition as a verifier and apply a confidence-threshold rule: if the verifier's reported confidence is at least *k*, accept the verdict; otherwise escalate to human review. For each (model, condition, threshold), we report:

- **Coverage**: fraction of instances auto-decided.
- **Auto-decision accuracy**: accuracy on the auto-decided subset.
- **Escalation rate**: complement of coverage.

This yields a coverage / accuracy trade-off curve per verifier. The full table is computed by scripts/threshold_policy_analysis.py and saved to results/threshold_policy.csv. A companion plot (figures/fig8_threshold_policy.{pdf,png}) shows accuracy as a function of coverage for each model and condition. The takeaway is that calibrated verifiers (notably Claude Sonnet 4) provide a usable trade-off where escalating low-confidence cases sharply increases auto-decision accuracy without escalating most of the workload, while overconfident verifiers (most GPT and Gemini conditions) cluster at high coverage with limited control over the trade-off.

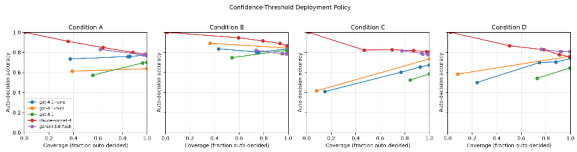


Figure 8: Confidence-threshold deployment policy, auto-decision accuracy vs. coverage for each model and condition.

Appendix G: Text-Reference Quality Ablation

To test whether the benefit of text references depends on their quality and specificity, we constructed three perturbed variants of each reference:

- **Short**: only the first sentence of the original reference.
- **Noisy**: the original reference plus a generic distractor sentence describing layout elements.
- **Wrong**: a deliberately off-topic reference that does not describe the task's expected outcome.

These variants are produced by scripts/generate_text_ref_variants.py (no API calls; deterministic) and the ablation is run by scripts/run_text_ref_ablation.py over a fixed-seed 100-instance subset, summarised by scripts/analyze_text_ref_ablation.py into results/textref_ablation_summary.csv. We evaluate GPT-4.1 Mini and Claude Sonnet 4, the model with the largest text-reference benefit and the most accurate verifier respectively. On this subset the majority-class (always-FAILURE) rate is 86.9%, and the full reference attains 84.8% accuracy / 0.595 F1 for GPT-4.1 Mini and 90.9% / 0.690 for Claude Sonnet 4. Results. Reference quality matters, and F1 rather than accuracy exposes it. Truncating the reference to its first sentence (short) costs GPT-4.1 Mini 0.231 F1 (0.595 → 0.364) and Claude Sonnet 4 0.157 F1 (0.690 → 0.533). Adding a distractor sentence (noisy) costs a further 0.031 and 0.019 F1 respectively, so verbosity is less damaging than omission. The wrong variant is the most instructive: accuracy rises to 86.0% (GPT-4.1 Mini) and 87.0% (Claude Sonnet 4), at or above the full-reference figure, while F1 collapses to 0.000 for both models. An off-topic reference does not merely fail to help; it drives the verifier to answer FAILURE on every instance, recovering the majority-class rate and losing all ability to recognise success. This is the same degenerate pattern seen in the SSIM baseline (Section 5.4), and it reinforces that accuracy alone is an unsafe headline metric on an 83.5%-FAILURE dataset. Practically, a verification pipeline that silently supplies a stale or mismatched reference would look accurate while having become useless.

Appendix H: Image Preprocessing and API Settings

To support reproducibility and to remove a common source of "why don't I get the same numbers?" questions, we document the exact preprocessing and API settings used for every verification call. The source of truth is visual-self-verify/llm_clients.py and config.py.

Screenshots. Final-state screenshots from VWA agent trajectories are 1280x2048 PNG (portrait, 5:8 aspect ratio). They are sent to each provider without any resizing or compression, base64-encoded. Reference screenshots from human trajectories are 800x446 JPEG (landscape; smaller because they were captured at a different stage of the VWA pipeline). The asymmetry between actual and reference resolutions is a property of the upstream VWA artifacts; we do not normalize them, so the reference contains less pixel-level detail than the actual screenshot. This is a relevant context for the visual-reference findings in Sections 5.1-5.3.

Provider settings.

Provider	Image format	Detail	Max output tokens	Temperature	Notes
OpenAI (gpt-4.1, mini, nano)	data URL base64, image_url block	detail: "high"	512	0	Direct API; provider may downscale internally above its native cap.
Anthropic (claude-sonnet-4)	source: base64, native PNG	n/a (native handling)			Anthropic auto-rescales to 1568px on the long edge if larger.
Google Gemini (3.6 flash)	inline_data parts, base64	n/a	4096	0.01	Gemini does not accept exact 0; we use the smallest legal value. Gemini 3.x charges internal reasoning tokens against max_output_tokens, so a 512-token cap

Provider	Image format	Detail	Max output tokens	Temperature	Notes
					truncated most responses mid-JSON; we raise the cap to 4096 and request response_mime_type=application/json. The visible answer remains ~80 tokens, comparable to the other providers.

Retries. Each call retries up to MAX_RETRIES = 3 with exponential backoff (1s, 2s, 4s). Persistent failures are logged as verdict: API_ERROR.

Parsing. Responses are parsed for the embedded JSON object via a five-step strategy (full JSON parse, fenced-block strip, regex search for the last {...verdict...}, brace-balanced fallback, fenced-anywhere). Persistent failures are logged as verdict: PARSE_ERROR. PARSE_ERROR rates and a bias check are reported in Appendix D.

Determinism. With temperature=0 (or 0.01 for Gemini), provider outputs are nominally deterministic but providers do not contractually guarantee bitwise reproducibility across versions. We mitigate by pinning model versions in config.py; per-call model_version strings are recorded in each result row so that retroactive drift checks are possible.

Total cost of the full study is reported in Section 5.6 ($35.04 across 10,560 calls). Reproducing the entire study costs less than the cost of a single human evaluator-day at typical contractor rates, this is the practical sense in which the recommended intervention is "low cost."

Appendix I: Full Bonferroni-Corrected Significance Table

The 30 paired McNemar tests (5 models × 6 condition pairs) are reported in full in results/S1_paired_tests.csv, with the matched instance IDs behind each test in results/S1_task_ids/. Bonferroni-significant pairs at α'=0.05/30=0.00167 (the same set survives Holm-Bonferroni step-down) are: GPT-4.1 Nano A→B; GPT-4.1 Mini A→B and A→D; GPT-4.1 A→B and B→C; Claude Sonnet 4 A→B. Four of the six are A→B comparisons, covering every model except Gemini 3.6 Flash, whose A→B change is not significant (p=0.450). Only one contrast involving visual references survives correction, GPT-4.1's B→C, so the earlier claim that visual references degrade most models does not hold at this significance threshold and is presented as suggestive in Section 5.1.

References

[1] Zhou, S., et al. "WebArena: A Realistic Web Environment for Building Autonomous Agents." *arXiv, 2024*. Available: https://arxiv.org/pdf/2307.13854

[2] He, H., et al. "WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models." *arXiv, 2024*. Available: https://arxiv.org/pdf/2401.13919

[3] Koh, J.Y., et al. "VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks." *arXiv, 2024*. Available: https://arxiv.org/pdf/2401.13649
[4] Zheng, L., et al. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena." *arXiv, 2023*. Available: https://arxiv.org/pdf/2306.05685
[5] Pan, J., et al. "Autonomous Evaluation and Refinement of Digital Agents." *arXiv preprint arXiv:2404.06474* (2024). Available: https://arxiv.org/abs/2404.06474
[6] Abuelsaad, T., et al. "Agent-E: From Autonomous Web Navigation to Foundational Design Principles in Browser Agents." *arXiv preprint arXiv:2407.13032*, *2024*. Available: https://arxiv.org/abs/2407.13032
[7] Zheng, B., et al. "GPT-4V(ision) is a Generalist Web Agent, if Grounded." *ICML'24: Proceedings of the 41st International Conference on Machine Learning, 2024*. Available: https://dl.acm.org/doi/10.5555/3692070.3694608
[8] Chen, X., et al. "Teaching Large Language Models to Self-Debug." *arXiv, 2024*. Available: https://arxiv.org/pdf/2304.05128
[9] Yue, X., et al. "MMMU: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark for Expert AGI." *arXiv, 2024*. Available: https://arxiv.org/pdf/2311.16502
[10] Liu, Y., et al. "MMBench: Is Your Multi-Modal Model an All-around Player?" *arXiv, 2024*. Available: https://arxiv.org/pdf/2307.06281
[11] Liu, X., et al. "AgentBench: Evaluating LLMs as Agents." *arXiv, 2024*. Available: https://arxiv.org/pdf/2308.03688
[12] Deng, X., et al. "Mind2Web: Towards a Generalist Agent for the Web." *arXiv (2023)*. Available: https://arxiv.org/pdf/2306.06070
[13] Xie, J., et al. "TravelPlanner: A Benchmark for Real-World Planning with Language Agents." *ICML'24: Proceedings of the 41st International Conference on Machine Learning, 2024*. Available: https://dl.acm.org/doi/10.5555/3692070.3694316
[14] Shinn, N., et al. "Reflexion: Language Agents with Verbal Reinforcement Learning." *arXiv 2023*. Available: https://arxiv.org/pdf/2303.11366
[15] Yao, S., et al. "ReAct: Synergizing Reasoning and Acting in Language Models." *arXiv, 2023*. Available: https://arxiv.org/pdf/2210.03629
[16] Kadavath, S., et al. "Language Models (Mostly) Know What They Know." *arXiv preprint arXiv:2207.05221* (2022). Available: https://arxiv.org/abs/2207.05221
[17] Guo, C, et al., "On Calibration of Modern Neural Networks." *arXiv, 2017*. Available: https://arxiv.org/pdf/1706.04599
[18] Lin, S., Hilton, J., Evans, O. "Teaching Models to Express Their Uncertainty in Words." *arXiv,* (2022). Available: https://arxiv.org/pdf/2205.14334
[19] Tian, K., et al. "Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback." *arXiv 2023*. Available: https://arxiv.org/pdf/2305.14975
[20] Naeini, M.P., Cooper, G.F., Hauskrecht, M. "Obtaining Well Calibrated Probabilities Using Bayesian Binning." *Twenty-Ninth AAAI Conference on Artificial Intelligence, 2015*. Available: https://ojs.aaai.org/index.php/AAAI/article/view/9602
[21] Wang, Z., et al., "Image Quality Assessment: From Error Visibility to Structural Similarity." *IEEE Transactions on Image Processing* 13.4 (2004): 600-612. Available: https://ieeexplore.ieee.org/document/1284395
[22] Yang, J., et al. "Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V." *arXiv preprint arXiv:2310.11441*, 2023. Available: https://arxiv.org/pdf/2310.11441
[23] Efron, B. "Bootstrap Methods: Another Look at the Jackknife." *Annals of Statistics* 7(1):1-26 *(1979)*. Available: https://projecteuclid.org/journals/annals-of-statistics/volume-7/issue-1/Bootstrap-Methods-Another-Look-at-the-Jackknife/10.1214/aos/1176344552.full
[24] McNemar, Q. "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages." *Psychometrika* 12(2):153-157 (1947). Available: https://link.springer.com/article/10.1007/BF02295996
[25] Li, J, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models." *arXiv, 2023*. Available: https://arxiv.org/pdf/2301.12597
[26] Wei, J., et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *arXiv, 2022*. Available: https://arxiv.org/pdf/2201.11903
[27] Cohen, J. "A Coefficient of Agreement for Nominal Scales." *Educational and Psychological Measurement* 20(1):37-46 *(1960)*. Available: https://journals.sagepub.com/doi/10.1177/001316446002000104
[28] Landis, J.R., Koch, G.G. "The Measurement of Observer Agreement for Categorical Data." Biometrics 33(1):159-174, 1977. Available: https://www.jstor.org/stable/2529310

Biography

Vishwak Thatikonda is an independent researcher and full-stack software engineer with over a decade of experience across distributed systems, federated GraphQL infrastructure, AWS serverless architecture, and AI-driven enterprise platforms. His recent applied work includes lead engineering on a mission-critical Master Data Management platform governing product, assortment, and store-attribute data for a global food-and-beverage retailer operating tens of thousands of locations. His work on the retailer's data foundation supports AI-driven personalization and inventory accuracy for its next-generation point-of-sale and mobile ordering systems. He holds a Master of Science in Computer Science from Arizona State University and a Bachelor of Science in Computer Science from Amrita University. Earlier in his career, he served as Lead Engineer at Kyte (USA), Senior Full-Stack Developer at Credit Sesame (USA), founding developer of the web application platform at AirWorks Solutions (USA), and contributed ground-segment software to the Mastcam-Z science payload for NASA's Mars 2020 Perseverance rover at Arizona State University's School of Earth and Space Exploration. He is the author of LambdaForge, an open-source serverless algorithmic trading engine built on AWS Lambda and EventBridge. He holds the AWS Certified Solutions Architect Associate certification.

Shravani Parsi received her M.S. in Software Engineering from San Jose State University, USA, and her B.Tech in Information Technology from VNR VJIET, India, where she graduated Summa Cum Laude as the Department Gold Medalist. She is currently a senior software engineer at a Fortune 500 retail technology company, where she leads Next.js and React architecture for a member-engagement platform and develops LLM- and LangGraph-based vision agents for automated UI testing. Her research interests include LLM agent orchestration and human-computer interaction in web systems.